

# DBMS Mastery

Database theory for the placement round  
normalization, ACID, serializability, indexing  
10 diagrams, 16 model answers, a recently-asked set, 45 FAQs

[@rathoreadiitya](#)

v1.0 · Updated 2026-06-06

Brand new to DBMS? Start at **§3 Start here: the student minimum** below, then read the core concepts in order.

This is DBMS theory, the part interviewers screen on alongside DSA: normalization, transactions, concurrency, indexing. Writing queries is a separate skill with its own practice. Read top to bottom if DBMS is new. Short on time? Use the contents to jump.

## Inside this sheet

---

1. Brand-new note + who this sheet is for
2. Inside this sheet (you are here)
3. Start here: the student minimum - what a DBMS round really tests
4. Why DBMS theory feels dry (and how to fix that)
5. DBMS vs file system
6. Three-schema architecture + data independence
7. Keys - super, candidate, primary, foreign, composite, alternate, surrogate
8. ER model - entities, attributes, relationships, cardinality
9. Functional dependencies + Armstrong's axioms + closure
10. Anomalies - why an un-normalized table hurts
11. Normalization - 1NF, 2NF, 3NF, BCNF, 4NF (worked)
12. Denormalization - when you undo it on purpose
13. Beyond the basics (advanced, not all fresher-required)
14. Transactions + ACID
15. Transaction states
16. Schedules + serializability (conflict vs view, precedence graph)
17. Recoverable + cascadeless schedules
18. Concurrency control - locks, 2PL, timestamp ordering, deadlock
19. Isolation levels + the anomaly grid
20. Indexing - dense vs sparse, B-tree vs B+ tree, clustered vs non-clustered
21. Comparison tables (the ones interviewers love)
22. Interview triggers - if they ask X, say Y
23. Common mistakes + traps
24. Model interview answers - say it like this (16 questions)
25. Recently asked in interviews - what current cycles hit
26. Interview FAQ bank - 45 questions
27. The 30-minute revision card
28. Final word

**If you have only 30 minutes - jump to §27 Revision card.**

# 3. Start here: the student minimum

A DBMS round for a fresher is not the whole textbook. It is a small core asked over and over. Learn this core cold and you will not drop easy points in most rounds.

*The five must-knows:*

1. Normalization - take a messy table to 1NF, 2NF, 3NF, and know BCNF. Be able to spot partial and transitive dependencies by eye.
2. ACID - what each of the four letters guarantees, with a one-line example.
3. The isolation grid - which of dirty read, non-repeatable read, and phantom read each isolation level allows.
4. Keys - primary vs candidate vs super vs foreign, and the difference between them in one line each.
5. Indexing basics - why an index speeds reads, clustered vs non-clustered, and what a B+ tree is.

**The one hands-on thing to drill: take a denormalized table (say an order sheet with customer name, product, and price repeated on every row), and normalize it step by step to 3NF on paper, naming the dependency you remove at each stage. If you can do that without notes, you are ahead of most candidates.**

Where to jump: normalization is \$11, ACID is \$14, the isolation grid is \$19, keys are \$7, indexing is \$20. Everything before \$13 is the fresher core. Everything after is depth that separates a good answer from a great one.

# 4. Why DBMS theory feels dry (and how to fix that)

Normalization and serializability sound like exam jargon because they are usually taught as rules to memorize. They are not rules. They are answers to real pain: data that contradicts itself, a booking that double-charges you, a report that reads half-finished money. Tie each idea to that pain and it sticks. That is what the Aha boxes in this sheet do - one everyday scene per concept.

# 5. DBMS vs file system

Before databases, applications stored data in flat files. That works until two programs need the same data, or you need to query it, or a crash strikes mid-write. A DBMS adds querying, concurrency control, integrity rules, security, and crash recovery on top of storage.

| Aspect      | Plain files                 | DBMS                         |
|-------------|-----------------------------|------------------------------|
| Redundancy  | Same data copied everywhere | Controlled by normalization  |
| Querying    | Write code by hand          | Declarative queries          |
| Concurrency | You manage locks yourself   | Built-in concurrency control |
| Integrity   | No enforced rules           | Constraints + keys enforced  |
| Recovery    | Lose data on crash          | Logs + recovery              |

💡 Aha - Files matlab ek register jismein sab apne haath se likhte hain - do log ek saath likhne lagein toh gadbad. DBMS woh manager hai jo register ke upar baitha hai: kaun kab likhega, koi rule toot na jaaye, aur crash ho toh aadha likha hua wapas theek ho jaaye, sab woh sambhalta hai.

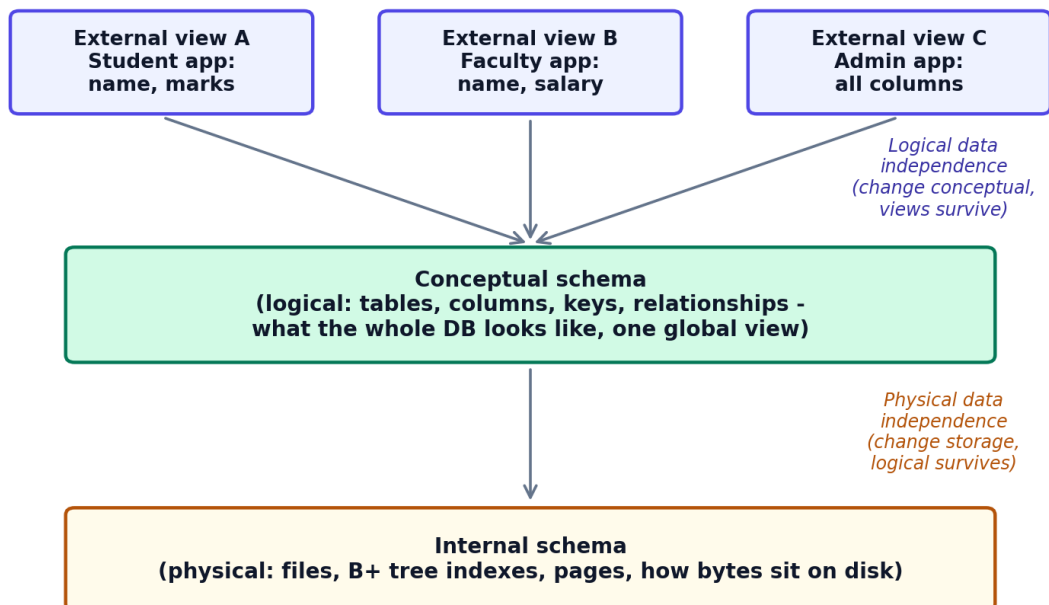
## 6. Three-schema architecture + data independence

ANSI-SPARC splits a database into three levels so that a change at one level does not force changes above it.

- Physical (internal) level - how data is actually stored on disk: files, indexes, blocks.
- Logical (conceptual) level - the tables, columns, and relationships; what data exists and how it links.
- View (external) level - what each user or app sees; a filtered slice of the logical schema.

Data independence is the payoff. Physical data independence: change the storage or add an index without touching the logical schema. Logical data independence: add a column or split a table without breaking existing views (harder to achieve in practice).

### Three-schema architecture - the same data, three views



*Three levels. A user sees a view; the view maps to the logical schema; the logical schema maps to physical storage.*

💡 Aha - Socho ek restaurant: physical level kitchen hai jahan actual khana banta hai, logical level pura menu hai jo sab dishes define karta hai, aur view level woh chhota menu card hai jo

tumhare table pe aata hai - kitchen badal do par customer ko farak nahi padta, yahi data independence hai.

## 7. Keys

A key is an attribute (or set) that identifies rows. The whole family comes from one idea: which columns let you pick out exactly one row.

| Key           | Meaning                                                             | Example                       |
|---------------|---------------------------------------------------------------------|-------------------------------|
| Super key     | Any set of columns that uniquely identifies a row (may have extras) | {roll_no, name}               |
| Candidate key | A minimal super key - no column can be removed                      | {roll_no}                     |
| Primary key   | The candidate key you pick as the main identifier; not null, unique | roll_no                       |
| Alternate key | A candidate key not chosen as primary                               | email (if roll_no is primary) |
| Composite key | A key made of two or more columns together                          | {order_id, product_id}        |
| Foreign key   | A column referencing another table's primary key                    | dept_id in Employee           |
| Surrogate key | An artificial id (auto-increment) with no business meaning          | id = 1, 2, 3...               |

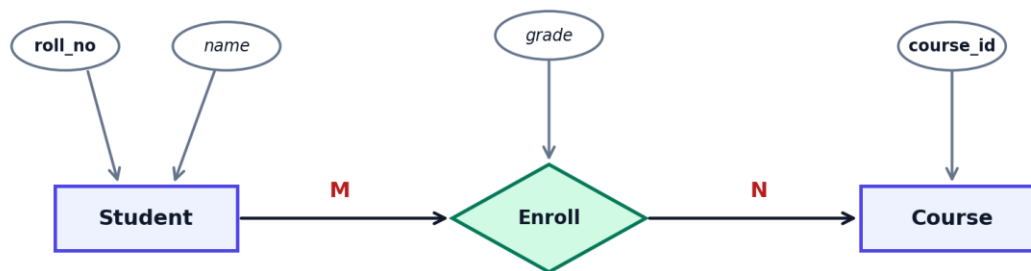
💡 Aha - Super key matlab itni saari pehchaan ki banda zaroor mil jaaye (naam + roll + photo), candidate key woh minimum pehchaan jisse kaam ban jaaye (sirf roll number), aur primary key woh ek jo tumne officially chuni. Foreign key doosri table ki primary key ko point karti hai, jaise tumhare form pe likha tumhara father ka Aadhaar number.

## 8. ER model

The Entity-Relationship model is how you design a schema before writing any table. Entities become tables, attributes become columns, relationships become foreign keys or link tables.

- Entity - a real thing you store (Student, Course). A weak entity cannot be identified without a parent (an Order Line needs its Order).
- Attributes - simple (age), composite (name = first + last), derived (age from date of birth), multivalued (multiple phone numbers).
- Relationship - an association (Student enrolls in Course).
- Cardinality - one-to-one, one-to-many, many-to-many. A many-to-many needs a junction table.

## ER model - Student M:N Course via Enroll (with cardinality)



*Underlined = primary key. Diamond = relationship. Ellipse = attribute.  
M:N means one student takes many courses AND one course has many students.*

*An ER fragment: Student and Course entities linked by a many-to-many Enrolls relationship, resolved with a junction table.*

💡 **Aha** - ER diagram tumhare database ka blueprint hai jaise ghar banane se pehle naksha. Entity matlab kamre (Student, Course), attribute matlab kamre ki cheezein (naam, age), aur relationship matlab darwaaze jo do kamron ko jodte hain - many-to-many ka matlab beech mein ek corridor (junction table) banana padega.

## ER to relational - the mapping rules

Interviewers ask you to convert an ER diagram to tables. The standard rules:

- Strong entity becomes a table; its attributes become columns, its key becomes the primary key.
- One-to-many - put a foreign key on the 'many' side.
- Many-to-many - make a separate junction table with foreign keys to both.
- Weak entity - a table with a foreign key to its owner plus its partial key as a composite primary key.
- Multivalued attribute (phone numbers) - move it to a separate table.
- Composite attribute - store each part as its own column.

💡 **Aha** - ER se table banane ka rule simple hai: har entity ek table, one-to-many mein foreign key 'many' wali side pe, aur many-to-many mein beech ek nayi junction table.

## 9. Functional dependencies + Armstrong's axioms

A functional dependency  $X \rightarrow Y$  means: if you know  $X$ , the value of  $Y$  is fixed. roll\_no  $\rightarrow$  name (one roll number, one name). FDs are the raw material of normalization - you normalize by removing the wrong kinds of dependency.

*Armstrong's axioms (sound and complete - every valid FD follows from these three):*

- Reflexivity - if  $Y$  is a subset of  $X$ , then  $X \rightarrow Y$  (trivial FD).
- Augmentation - if  $X \rightarrow Y$ , then  $XZ \rightarrow YZ$  (add the same thing to both sides).
- Transitivity - if  $X \rightarrow Y$  and  $Y \rightarrow Z$ , then  $X \rightarrow Z$ .

Attribute closure ( $X^+$ ) is the set of all attributes you can determine from  $X$ . If  $X^+$  covers every attribute,  $X$  is a super key. That is how you find candidate keys from a set of FDs.

💡 **Aha - Functional dependency matlab agar tumhe roll number pata hai toh naam apne aap pata chal jaata hai - ek determine karta hai doosre ko. Closure nikalna matlab ek pehchaan se poochho 'isse aur kya kya pata chal sakta hai', aur agar poori table pata chal gayi toh woh super key hai.**

## 10. Anomalies - why an un-normalized table hurts

Cramming everything into one wide table causes three anomalies. Normalization exists to kill them.

| Anomaly | What goes wrong                                                                                                  |
|---------|------------------------------------------------------------------------------------------------------------------|
| Insert  | You cannot add a fact without another unrelated fact (cannot add a course with no student enrolled yet).         |
| Update  | One fact stored in many rows; update one row and the others now disagree (price changed in one order line only). |
| Delete  | Deleting one row wipes out an unrelated fact (removing the last student deletes the course itself).              |

💡 **Aha - Socho ek hi register mein student, course aur teacher sab ek line mein likhe ho. Teacher ka phone number badla toh har line mein badalna padega (update anomaly), aur agar last student ne course chhod diya toh poora course hi register se gaayab (delete anomaly). Isliye alag alag tables banate hain.**

## 11. Normalization - 1NF to 4NF

Normalization splits tables so each fact lives in exactly one place. Each normal form removes one specific kind of bad dependency. Go in order: you cannot be in 3NF without first being in 2NF.

**Normalization ladder - each step removes one kind of redundancy**



*Each form ASSUMES the previous one. A table in 3NF is already in 2NF and 1NF.*

*The ladder: 1NF removes repeating groups, 2NF removes partial dependency, 3NF removes transitive dependency, BCNF tightens 3NF, 4NF removes multi-valued dependency.*

## 1NF - atomic values

Every cell holds a single value. No lists, no repeating groups. A phones column holding '99999, 88888' breaks 1NF; split it into rows or a separate table.

💡 **Aha - 1NF ka matlab: ek dabbe mein ek hi cheez. Tiffin ke ek khaane mein daal aur sabzi dono mat thoso - har cheez ka apna khaana.**

## 2NF - no partial dependency

Be in 1NF, and no non-key column may depend on only PART of a composite key. If the key is {order\_id, product\_id} and product\_name depends only on product\_id, that is a partial dependency - move product\_name to a Product table. 2NF only matters when the primary key is composite.

💡 **Aha - 2NF tabhi aata hai jab key do cheezon se bani ho. Agar koi column sirf aadhi key pe depend kar raha hai - jaise product ka naam sirf product\_id pe, order\_id se koi lena dena nahi - toh usse alag table mein bhejo.**

## 3NF - no transitive dependency

Be in 2NF, and no non-key column may depend on another non-key column. If Student has roll\_no -> dept\_id -> dept\_name, then dept\_name depends on dept\_id (a non-key), which is transitive. Move dept\_name to a Department table keyed by dept\_id.

💡 **Aha - 3NF ka matlab: har non-key column sirf primary key ko jawab de, kisi doosre non-**

key ko nahi. dept\_name ko roll\_no se poochho, dept\_id ke through ghuma ke mat poochho - warna department ka naam badla toh har student row mein badalna padega.

## BCNF - every determinant is a candidate key

A stricter 3NF. For every non-trivial FD  $X \rightarrow Y$ ,  $X$  must be a super key. BCNF catches the rare case 3NF misses: when a non-prime attribute determines part of a candidate key (overlapping candidate keys). Most well-designed 3NF tables are already in BCNF.

💡 **Aha - BCNF matlab: jo bhi banda kisi cheez ko determine kar raha hai, woh khud ek poori key honi chahiye - aadhi-adhuri pehchaan se kisi key ka hissa decide nahi hona chahiye.**

One nuance worth a sentence: a 3NF decomposition can always keep all functional dependencies (dependency-preserving), but a BCNF decomposition sometimes cannot. When forcing BCNF would lose a dependency you need to enforce, you stop at 3NF and accept the one small anomaly.

## 4NF - no multi-valued dependency

Be in BCNF, and remove multi-valued dependencies: when one key independently determines two separate multi-valued facts (a person's skills and their languages), do not store them in one table as a cross-product. Split into two tables.

💡 **Aha - 4NF: agar ek banda ke do alag alag multi-valued facts hain - jaise uski skills aur uski languages - inko ek table mein mila ke har combination mat banao, do alag tables banao.**

Interview line that lands: 1NF kills repeating groups, 2NF kills partial dependency, 3NF kills transitive dependency, BCNF tightens 3NF for overlapping keys, 4NF kills multi-valued dependency. Say them in that order and you sound fluent.

# 12. Denormalization - when you undo it on purpose

Normalization removes redundancy but adds joins. On a read-heavy system, joining five tables for every page load is slow. Denormalization deliberately re-introduces some redundancy (a cached total, a copied column) to cut joins and speed reads, accepting the extra update work. It is a trade-off, not a mistake.

💡 **Aha - Denormalization matlab jaanboojh ke thoda data repeat karna taaki har baar join na**

karna pade - jaise hostel ke gate pe ek copy notice chipka dena instead of har bande ko warden ke office bhejna. Padhne mein fast, par update karte waqt dono jagah badalna padega.

## 13. Beyond the basics (advanced, not all fresher-required)

Everything from here - transactions, serializability, 2PL, timestamp ordering - is depth for senior rounds and backend projects. For most fresher interviews it is recognise-the-words, not memorise-every-detail. The ONE exception worth truly knowing cold: the isolation-level anomaly grid in §19. That one comes up constantly. Read the rest once so the terms are not strangers.

## 14. Transactions + ACID

A transaction is a group of operations treated as one unit - a money transfer is debit plus credit, and both must happen or neither. ACID is the four guarantees a DBMS gives every transaction.

| Property    | Guarantee                                                         | One-line example                                                      |
|-------------|-------------------------------------------------------------------|-----------------------------------------------------------------------|
| Atomicity   | All operations happen, or none do (rollback on failure)           | Debit fails after credit, so the whole transfer rolls back            |
| Consistency | Moves the DB from one valid state to another, keeping constraints | Total money before = total money after                                |
| Isolation   | Concurrent transactions do not see each other's half-done work    | Two transfers at once give the same result as running them one by one |
| Durability  | Once committed, changes survive a crash                           | Power cut after commit, money is still moved                          |

## ACID - the four promises every transaction makes



*Bank transfer = the classic example: debit + credit must both happen (A), balance never goes negative (C).*

*The four ACID guarantees on one transaction.*

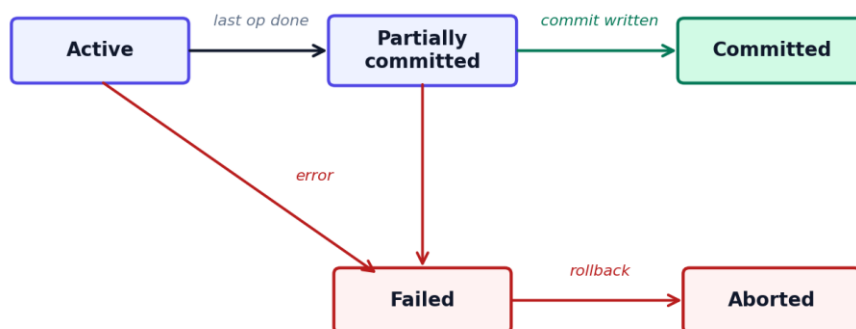
💡 **Aha - ACID ko shaadi se yaad rakho:** Atomicity matlab dono 'haan' bolein ya shaadi cancel (aadhi shaadi nahi hoti), Consistency matlab saari rasmein theek se पूरी hon, Isolation matlab do shaadiyan ek hall mein ho rahi hon toh ek doosre mein ghusein nahi, aur Durability matlab register pe sign ho gaya toh light chali bhi jaaye toh shaadi pakki.

## 15. Transaction states

A transaction moves through a fixed lifecycle. It runs (active), finishes its last step (partially committed), and then either makes it permanent (committed) or hits an error and rolls back (failed, then aborted).

- Active - executing its operations.
- Partially committed - final statement done, changes not yet on disk.
- Committed - changes made permanent and durable.
- Failed - an error stopped normal execution.
- Aborted - rolled back to the state before it started.

## Transaction states - the lifecycle of one transaction



*Committed and Aborted are the only two terminal (final) states.*

*Transaction lifecycle: active to partially committed to committed, or via failed to aborted.*

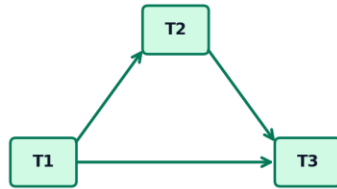
💡 **Aha - Transaction states matlab ek online order ka safar: active (cart bhar rahe ho), partially committed (pay kar diya), committed (order confirm, ab cancel nahi), ya failed-aborted (payment fail, sab wapas cart mein).**

## 16. Schedules + serializability

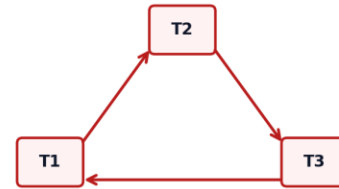
When transactions run concurrently, the DBMS interleaves their operations into a schedule. A serial schedule runs them one fully after another - always correct but slow. A concurrent schedule interleaves them for speed, but must still produce a result equal to SOME serial order. That property is serializability.

Conflict serializability is the practical test. Two operations conflict when they touch the same data item, belong to different transactions, and at least one is a write. Draw a precedence graph (a node per transaction, an edge  $T_i \rightarrow T_j$  when an operation of  $T_i$  must come before a conflicting one of  $T_j$ ). If the graph has NO cycle, the schedule is conflict-serializable.

Conflict serializability - draw an edge  $T_i \rightarrow T_j$  on a conflicting pair, then look for a cycle  
No cycle = serializable      Cycle = NOT serializable



Topological order  $T1 \rightarrow T2 \rightarrow T3$  exists, so an equivalent serial schedule exists.



$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$  is a cycle. No serial order works, so the schedule is not serializable.

A precedence graph. No cycle means conflict-serializable; a cycle means it is not.

View serializability is a looser superset: every conflict-serializable schedule is view-serializable, but not the reverse. It is harder to test (NP-hard) so conflict serializability is what interviews use.

💡 Aha - Serializability matlab bhale operations aage peeche mix ho jaayein, final result aisa lage jaise transactions ek ke baad ek chale the. Precedence graph mein agar cycle ban gaya - A, B ka wait kare aur B, A ka - toh order possible nahi, schedule galat.

## 17. Recoverable + cascadeless schedules

A recoverable schedule never lets a transaction commit before the one whose data it read has committed - otherwise a rollback would strand already-committed work. A cascadeless schedule goes further: a transaction only reads values written by already-committed transactions, so one abort never forces a chain of other rollbacks (cascading abort).

💡 Aha - Cascadeless matlab kisi ka adhura kaam padh ke aage mat badho. Agar tumne dost ke kachche notes se padh liya aur usne notes badal diye, toh tumhe bhi dobara padhna padega - isliye sirf final, committed notes se hi padho.

## 18. Concurrency control

Concurrency control is how the DBMS keeps interleaved transactions serializable. The two big families are locking and timestamps.

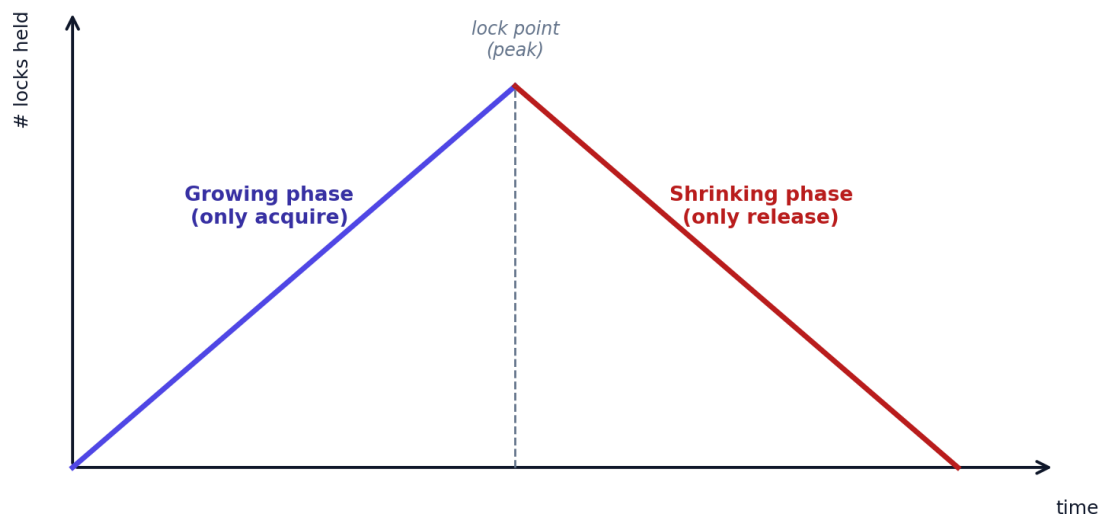
### Lock-based + Two-Phase Locking (2PL)

Locks come in two modes: shared (S, for reading, many allowed) and exclusive (X, for writing, one only). Two-Phase Locking splits a transaction's locking into two phases:

- Growing phase - it may acquire locks but release none.
- Shrinking phase - once it releases its first lock, it may release more but acquire none.

Basic 2PL guarantees conflict-serializability but NOT deadlock-freedom. Strict 2PL holds all exclusive (write) locks until commit/abort, which prevents cascading aborts. Rigorous 2PL holds ALL locks until commit.

### Two-phase locking (2PL) - acquire all locks, THEN release



Rule: once a txn releases its FIRST lock, it can never acquire another. This guarantees conflict-serializability.

2PL: locks only go up in the growing phase, only come down in the shrinking phase.

## Timestamp ordering

Each transaction gets a timestamp at start. Operations are forced into timestamp order using per-item read and write timestamps; a transaction that arrives too late to fit the order is aborted and restarted. No locks, so no deadlock, but more restarts.

## Deadlock handling

Locking can deadlock (T1 holds A waiting for B, T2 holds B waiting for A). Prevention schemes use timestamps:

- Wait-Die - an older transaction waits for a younger one; a younger one requesting an older's lock dies (restarts).
- Wound-Wait - an older transaction wounds (preempts) a younger one; a younger one waits.

The alternative to prevention is detection: let deadlocks happen, build a wait-for graph, and if it has a cycle, abort one transaction (the victim) to break it. Most production databases use detection because it wastes fewer transactions than strict prevention.

💡 Aha - 2PL ko ek shopping trip socho: pehle saara saaman utha lo (growing phase), ek baar

wapas rakhna shuru kiya toh phir kuch naya nahi utha sakte (shrinking phase). Strict 2PL matlab bill (commit) hone tak trolley se kuch chhodo mat, taaki koi tumhara adhura saaman utha ke bhaag na jaaye.

## 19. Isolation levels + the anomaly grid

Full isolation (Serializable) is correct but slow, so SQL defines weaker levels that allow some anomalies for speed. Know the three anomalies first:

- Dirty read - you read another transaction's uncommitted change, which may be rolled back.
- Non-repeatable read - you read a row twice and get different values because another transaction updated and committed in between.
- Phantom read - you run the same range query twice and get a different SET of rows because another transaction inserted or deleted matching rows.

*The grid every interviewer wants (allowed = the anomaly can still happen at that level):*

| Isolation level  | Dirty read | Non-repeatable read | Phantom read |
|------------------|------------|---------------------|--------------|
| Read Uncommitted | Allowed    | Allowed             | Allowed      |
| Read Committed   | Prevented  | Allowed             | Allowed      |
| Repeatable Read  | Prevented  | Prevented           | Allowed      |
| Serializable     | Prevented  | Prevented           | Prevented    |

**Isolation levels vs anomalies (ANSI SQL standard)**

| Isolation level  | Dirty read | Non-repeatable | Phantom   |
|------------------|------------|----------------|-----------|
| Read Uncommitted | Possible   | Possible       | Possible  |
| Read Committed   | No         | Possible       | Possible  |
| Repeatable Read  | No         | No             | Possible* |
| Serializable     | No         | No             | No        |

\* MySQL InnoDB prevents phantoms at Repeatable Read via next-key locks - a real-world nuance.

*The isolation-anomaly grid. Higher levels prevent more anomalies at the cost of concurrency.*

💡 Aha - Isolation levels ko theek se yaad rakho: Read Uncommitted matlab tum kisi ka adhura, bina submit kiya kaam bhi padh lete ho (sabse kachcha). Jaise jaise level upar jaata hai, DBMS zyada strict hota jaata hai, aur Serializable pe aakar lagta hai sab ek ke baad ek hi chal rahe the - sabse safe, sabse slow.

How each level actually does it:

- Read Uncommitted - no read locks, so it prevents nothing.
- Read Committed - a read only sees committed values, killing dirty reads.
- Repeatable Read - holds read locks on rows it touched until commit, so a re-read is stable (but new rows can still appear).
- Serializable - locks the query range itself, so even phantom inserts are blocked.

## 20. Indexing

An index is a side structure that lets the DBMS find rows without scanning the whole table - like the index at the back of a book. It speeds reads and slows writes (the index must be updated too).

- Dense index - one entry per row.
- Sparse index - one entry per block, possible only when the data is sorted on the key.

### B-tree vs B+ tree

Both are balanced multi-way trees that keep lookups at  $O(\log n)$ . The difference is where the data lives. A B+ tree keeps all records in the leaf nodes only and links the leaves together, so range scans walk the leaf chain. A B-tree stores data in internal nodes too, so range scans are harder. Databases use B+ trees by default.

**B-tree: keys + data in every node**



*Data record can sit in an internal node.  
No links between leaves -> range scan  
must climb up and down.*

**B+ tree: data only in leaves, leaves linked**



*Internal nodes are index-only (copies of keys).  
Linked leaves -> range scan is one walk  
along the bottom. Why DBs prefer B+ tree.*

*B-tree keeps data in internal nodes; B+ tree keeps data only in linked leaves, which is why range scans are fast.*

### Clustered vs non-clustered

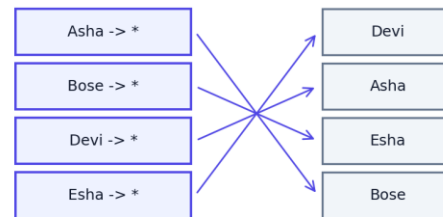
A clustered index stores the table rows physically in index-key order, so there is at most ONE per table (the rows can only be sorted one way). A non-clustered index is a separate structure holding the key plus a pointer to the row; you can have many per table.

### Clustered index: table sorted by the key

|      |      |
|------|------|
| id=1 | Asha |
| id=2 | Bose |
| id=3 | Devi |
| id=4 | Esha |

Rows are physically stored in key order.  
Only ONE per table (data can sit in one order).

### Non-clustered index: separate list of pointers



Index sorted; table (heap) unsorted.  
MANY allowed per table.

Clustered = the table itself is sorted by the key. Non-clustered = a separate lookup that points back to rows.

💡 Aha - Index matlab kitaab ke peeche wala index - poori kitaab padhe bina page mil jaata hai. Clustered index matlab kitaab khud alphabetical order mein chhapi hai (sirf ek tarah se sort ho sakti hai), non-clustered matlab alag se ek list jo bolti hai 'yeh topic page 40 pe hai' - aisi kai lists ban sakti hain.

## Hash index

A hash index hashes the key to find a row in  $O(1)$  for exact-match lookups (WHERE id = 5). It cannot do range queries (WHERE id > 5) or sorting, because hashing scatters order. Use it when every query is an exact match.

💡 Aha - Hash index matlab dictionary - exact shabd do, turant mil jaata hai. Par 'a se shuru hone wale saare shabd' maango toh hash index bekaar, uske liye B+ tree chahiye.

## Bonus for backend / SDE rounds (recognise the words)

Not fresher-core, but worth a line each so a backend interviewer does not catch you blank:

- CAP theorem - a distributed store can guarantee only two of Consistency, Availability, Partition tolerance at once.
- ACID vs BASE - single-node SQL gives ACID; many NoSQL stores trade it for BASE (Basically Available, Soft state, Eventually consistent).
- Sharding vs replication - sharding splits rows across servers by a key (more throughput); replication copies data across servers (redundancy + read scaling).
- Write-ahead log (WAL) + checkpoint - changes are logged to disk before the data is written, so a crash can replay the log from the last checkpoint. This is how Durability actually works.

- MVCC (optimistic concurrency) - each transaction reads a snapshot instead of locking, and checks for conflicts only at commit. Used by PostgreSQL.

💡 Aha - Yeh sab backend round ke liye hai: CAP matlab distributed mein teen mein se do hi mil sakte hain, sharding matlab data alag alag servers pe baant do, aur WAL matlab har kaam pehle diary mein likho taaki crash ke baad replay kar sako.

## 21. Comparison tables (the ones interviewers love)

### DELETE vs TRUNCATE vs DROP

|                | DELETE              | TRUNCATE   | DROP            |
|----------------|---------------------|------------|-----------------|
| Removes        | Chosen rows (WHERE) | All rows   | The whole table |
| Structure kept | Yes                 | Yes        | No              |
| Rollback       | Yes (logged)        | Usually no | No              |
| Type           | DML                 | DDL        | DDL             |

### Primary key vs Unique key

| Aspect          | Primary key       | Unique key                           |
|-----------------|-------------------|--------------------------------------|
| Nulls           | Not allowed       | One null allowed (usually)           |
| Count per table | Exactly one       | Many                                 |
| Purpose         | Main row identity | Enforce uniqueness on another column |

### Shared lock vs Exclusive lock

| Aspect             | Shared (S)    | Exclusive (X)                 |
|--------------------|---------------|-------------------------------|
| For                | Reading       | Writing                       |
| Concurrent holders | Many          | One                           |
| Blocks             | Other writers | All other readers and writers |

## 22. Interview triggers - if they ask X, say Y

| If they ask / hint                             | Reach for                                                   |
|------------------------------------------------|-------------------------------------------------------------|
| Same data stored in many rows, update mismatch | Normalization - this is an update anomaly, normalize to 3NF |
| Composite key, column depends on part of it    | 2NF violation - partial dependency                          |
| Non-key column depends on another non-key      | 3NF violation - transitive dependency                       |
| Money transfer, both steps or neither          | Transaction + Atomicity                                     |
| Two users updating the same row at once        | Concurrency control - locking or 2PL                        |
| Read got a value that later vanished           | Dirty read - raise the isolation level                      |

| If they ask / hint                     | Reach for                                     |
|----------------------------------------|-----------------------------------------------|
| Same query, different rows second time | Phantom read - needs Serializable             |
| Query slow on a big table scan         | Add an index (then mention B+ tree)           |
| Need range queries fast                | B+ tree (linked leaves)                       |
| Deadlock between two transactions      | Wait-Die or Wound-Wait, or deadlock detection |

## 23. Common mistakes + traps

| Wrong thinking                              | Correct thinking                                                        |
|---------------------------------------------|-------------------------------------------------------------------------|
| 2NF applies to every table                  | 2NF only matters when the primary key is composite                      |
| 3NF and BCNF are the same                   | BCNF is stricter; it catches overlapping-candidate-key cases 3NF misses |
| Normalization always improves performance   | It removes redundancy but adds joins; reads can get slower              |
| A primary key can be null                   | Never - primary key is not null and unique                              |
| More indexes are always better              | Indexes speed reads but slow every insert, update, and delete           |
| A table can have many clustered indexes     | At most one - rows have only one physical order                         |
| 2PL prevents deadlock                       | 2PL guarantees serializability, not deadlock-freedom                    |
| Serializable is the default isolation level | Most databases default to Read Committed                                |
| Foreign key must reference a primary key    | It references a candidate key (usually the primary key)                 |

## 24. Model interview answers - say it like this

The 45-question bank below is for rapid recall. This section is the opposite: the 16 questions a DBMS round actually opens with, each with a full answer you can speak in 20 to 40 seconds. Every answer follows the same shape - define it in one line, give one concrete example, then handle the follow-up before they ask. Read these out loud once; that is what makes them stick.

**How to use a model answer: lead with the one-line definition so the interviewer knows you know it, then ground it in ONE example, then stop. Rambling past the example is how strong answers turn average. The follow-up line is the question they ask next - have it ready and you look two steps ahead.**

### Q. What is a DBMS, and why use one over plain files?

**Say this:** A DBMS is software that stores, retrieves, and manages data while handling the hard parts for you: concurrency control so many users can read and write at once, integrity constraints so bad data is rejected, transactions so partial updates never corrupt state, and recovery so a crash does not lose committed work. With plain files you would hand-write all of that, and there is no easy querying, no enforced relationships, and heavy data redundancy.

**Follow-up they hit you with:** *DBMS vs RDBMS? An RDBMS is a DBMS that stores data as related tables with keys and supports SQL; relational is the most common kind of DBMS, not a different thing.*

### Q. Explain the different types of keys.

**Say this:** A super key is any set of columns that uniquely identifies a row. A candidate key is a minimal super key, where no column can be dropped without losing uniqueness. The primary key is the one candidate key you pick as the main identifier, and it is not null and unique. Any other candidate key is an alternate key. A foreign key is a column that references another table's candidate key to enforce referential integrity. A composite key is just a key made of more than one column.

**Weak answer (skip it):** *Saying a primary key and a unique key are identical. A primary key is not null and there is one per table; a unique key can hold one null and you can have several per table.*

**Follow-up they hit you with:** *Can a foreign key be null? Yes, unless declared NOT NULL - a null foreign key just means the relationship is not set yet.*

### Q. What is normalization and why do we do it?

**Say this:** Normalization is organising tables to remove redundancy by splitting data into related tables linked by keys. The reason is to kill the three anomalies that redundancy causes: an insert anomaly, where you cannot add one fact without another; an update anomaly, where the same value stored in many rows can go out of sync; and a delete anomaly, where removing one row loses unrelated information. We normalise step by step through 1NF, 2NF, 3NF, and BCNF.

**Follow-up they hit you with:** *Walk the forms: 1NF is atomic values, 2NF removes partial dependency on part of a composite key, 3NF removes transitive dependency on a non-key column, and BCNF tightens 3NF so every determinant is a super key.*

### Q. Difference between 3NF and BCNF?

**Say this:** Both deal with dependencies, but BCNF is stricter. A table is in 3NF if, for every functional dependency X gives Y, either X is a super key or Y is part of a candidate key. BCNF drops that second escape clause: for every non-trivial dependency X gives Y, X must be a super key, full stop. So every BCNF table is in 3NF, but a 3NF table can violate BCNF when a non-prime attribute determines part of a candidate key.

**Follow-up they hit you with:** Is BCNF always achievable without losing dependencies? No. Sometimes you cannot reach BCNF while keeping all functional dependencies, so 3NF is the highest form that always guarantees a lossless, dependency-preserving decomposition.

### Q. When would you denormalize on purpose?

**Say this:** Denormalization is deliberately adding back redundancy to speed up reads. A fully normalised schema needs many joins to answer a query, and on a read-heavy system those joins get expensive. So you might store a computed total or duplicate a frequently joined column to cut the joins. The cost is that writes now have to update the duplicated data in more than one place, so you only do it when reads massively outnumber writes and you have measured the join as the bottleneck.

**Follow-up they hit you with:** How do you keep denormalized data consistent? With triggers, application logic, or scheduled jobs that refresh the redundant copies, accepting that they can be briefly stale.

### Q. Explain ACID properties with an example.

**Say this:** ACID is the four guarantees a transaction gives. Take a bank transfer of 100 from A to B. Atomicity: both the debit and the credit happen, or neither does, so money never vanishes mid-way. Consistency: the transaction moves the database from one valid state to another, so the total balance is preserved. Isolation: if two transfers run at once, the result is as if they ran one after another. Durability: once it commits, the change survives a crash because it is written to the log.

**Follow-up they hit you with:** How is durability actually implemented? Write-ahead logging: the change is written to a log on stable storage before commit, so recovery can replay it after a crash.

### Q. What is a transaction and what are its states?

**Say this:** A transaction is a sequence of operations treated as a single all-or-nothing logical unit of work. It moves through states: it starts Active while executing, becomes Partially Committed after its last operation but before the commit is durable, then Committed once changes are saved. If something fails it goes to the Failed state and then Aborted, where it is rolled back to the state before it began.

**Follow-up they hit you with:** What happens to the data on abort? It is rolled back using the log, so the database looks exactly as it did before the transaction started.

### Q. What is serializability and how do you test for it?

**Say this:** A concurrent schedule is serializable if its outcome is equal to some serial schedule, that is, as if the transactions ran one after another with no interleaving. That is the gold standard for correctness under concurrency. The practical, testable version is conflict serializability: you build a precedence graph with a node per transaction and an edge for each conflicting pair of operations, and if that graph has no cycle, the schedule is conflict-serializable.

**Weak answer (skip it):** Saying any interleaved schedule is fine as long as each transaction is correct alone. Interleaving can produce results no serial order ever would; that is exactly what serializability rules out.

**Follow-up they hit you with:** Conflict vs view serializability? View is a looser superset that allows a few more schedules, but testing it is NP-hard, so conflict serializability is what we actually check.

### Q. What is two-phase locking, and does it prevent deadlock?

**Say this:** Two-phase locking, 2PL, is a protocol that guarantees conflict serializability. Each transaction has a growing phase where it only acquires locks and a shrinking phase where it only releases them; once it releases its first lock it can never acquire another. That ordering forces a serializable schedule. But it does NOT prevent deadlock: two transactions can each hold a lock the other wants and wait forever.

**Weak answer (skip it):** Saying 2PL prevents deadlock. It guarantees serializability only; deadlock is still possible and is handled separately.

**Follow-up they hit you with:** What does strict 2PL add? It holds all exclusive locks until commit, which prevents cascading aborts and gives recoverable schedules.

### Q. Shared vs exclusive locks?

**Say this:** These are the two lock modes for concurrency control. A shared lock is taken for reading, and many transactions can hold a shared lock on the same item at once, because concurrent reads do not conflict. An exclusive lock is taken for writing, and only one transaction can hold it, with no other shared or exclusive lock allowed at the same time, because a write conflicts with everything. So reads share, writes are exclusive.

**Follow-up they hit you with:** What is lock upgrading? A transaction holding a shared lock can upgrade to exclusive when it decides to write, provided no other transaction holds a shared lock on that item.

### Q. What are isolation levels and the anomalies they allow?

**Say this:** Isolation levels trade correctness for concurrency by controlling which read anomalies are possible. Read Uncommitted allows dirty reads, non-repeatable reads, and phantoms. Read Committed stops dirty reads but still allows non-repeatable reads and phantoms. Repeatable Read stops non-repeatable reads, leaving only phantoms. Serializable allows none of them. Higher isolation means more locking and less concurrency, so you pick the lowest level your correctness needs.

**Follow-up they hit you with:** What is a phantom read exactly? You run the same range query twice and the second run returns extra rows another transaction inserted in between, even though your existing rows did not change.

### Q. Dirty read vs non-repeatable read vs phantom read?

**Say this:** All three are concurrency anomalies, getting progressively subtler. A dirty read is reading another transaction's uncommitted change that might still be rolled back. A non-repeatable read is reading the same row twice and getting different values because another transaction updated and committed it in between. A phantom read is running the same range query twice and getting a different set of rows because another transaction inserted or deleted matching rows.

**Follow-up they hit you with:** Which isolation level kills which? Read Committed kills dirty reads, Repeatable Read kills non-repeatable reads, Serializable kills phantoms too.

### Q. What is an index and how does it speed up queries?

**Say this:** An index is a separate data structure, usually a B+ tree, that stores the indexed column's values in sorted order with pointers back to the rows. Instead of scanning every row to find a match, the database walks the tree in logarithmic time, so lookups and range scans on that column get far faster. The cost is that every insert, update, and delete must also maintain the index, and it takes extra storage, so you index the columns you actually search and join on, not every column.

**Follow-up they hit you with:** Why a B+ tree and not a B-tree? A B+ tree keeps all data in the leaves and links the leaves together, so range scans just walk the leaf chain, which is exactly what queries need.

### Q. Clustered vs non-clustered index?

**Say this:** A clustered index determines the physical order of the rows on disk, so the table itself is sorted by that key, and you get at most one per table because rows can only be stored in one order. A non-clustered index is a separate structure holding the sorted key values with pointers to the rows, so you can have many of them per table. A clustered index is faster for range queries on its key; a non-clustered index needs an extra hop to fetch the actual row.

**Weak answer (skip it):** Saying a table can have several clustered indexes. Only one - the rows have a single physical order.

**Follow-up they hit you with:** In many databases the primary key automatically becomes the clustered index, which is why looking up by primary key is so fast.

### Q. What is a functional dependency and how is it used?

**Say this:** A functional dependency X gives Y means that if you know the value of X, the value of Y is fixed - for example a roll number determines a student's name. Functional dependencies are the foundation of normalization: you use them to find candidate keys by computing attribute closures, and you check each normal form against them. A determinant that is not a super key is exactly what BCNF forbids.

**Follow-up they hit you with:** What is attribute closure? Starting from a set of attributes, it is everything you can derive using the functional dependencies; if the closure covers all attributes, that set is a super key.

### Q. What is the difference between DELETE, TRUNCATE, and DROP?

**Say this:** All three remove data but at different levels. DELETE is a DML command that removes rows one by one based on a WHERE clause, it is logged, and it can be rolled back. TRUNCATE removes all rows at once by deallocating the data pages, it is much faster, minimally logged, and resets identity counters, but it removes every row. DROP is DDL: it deletes the entire table, its structure, indexes, and constraints, so the table no longer exists.

**Weak answer (skip it):** Saying TRUNCATE and DELETE are the same. DELETE is row-by-row and filterable; TRUNCATE wipes the whole table fast and resets identity.

**Follow-up they hit you with:** Can you roll back a TRUNCATE? It depends on the database - in some it is transactional and rolls back, in others it is not, so treat it as not easily reversible.

@rathoreaditiya

## 25. Recently asked in interviews - what current cycles hit

These keep coming up in 2025-26 placement and off-campus rounds, grouped by where they land. Same syllabus, asked the way real interviewers ask it. If a row feels shaky, go back to that section before your round.

| Where it shows up     | What they actually ask                                                                                                             |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Keys + normalization  | Super vs candidate vs primary key. Normalize this table to 3NF step by step. 3NF vs BCNF with an example where they differ.        |
| ACID + transactions   | Explain ACID on a bank-transfer. What the transaction states are. What atomicity guarantees if the server crashes mid-transaction. |
| Concurrency           | What 2PL is and whether it gives serializability. Shared vs exclusive locks. How a database detects and breaks a deadlock.         |
| Isolation + anomalies | The four isolation levels and the anomaly each one stops. Dirty vs non-repeatable vs phantom read with an example.                 |
| Indexing              | Why a B+ tree over a B-tree for an index. Clustered vs non-clustered. When an index actually slows you down.                       |
| SQL query round       | Second highest salary. Delete duplicate rows keeping one. Nth highest without LIMIT. DELETE vs TRUNCATE vs DROP. CHAR vs VARCHAR.  |

💡 Aha - Dhyaan do: theory round in 6 areas pe ghoomta hai, aur practical round seedha SQL likhwata hai. Dono ke liye alag taiyari chahiye.

### The practical round is just SQL queries

Most DBMS interviews end with a laptop and a schema: write the query. These six on LeetCode are the exact ones that get asked, mapped to the row above. If you can write all six cold, the SQL round is handled.

- [175. Combine Two Tables](#) - LEFT JOIN basics, keep rows with no match
- [176. Second Highest Salary](#) - the classic, handle the no-second-row case
- [177. Nth Highest Salary](#) - generalise it without LIMIT hardcoding
- [182. Duplicate Emails](#) - GROUP BY + HAVING COUNT > 1
- [196. Delete Duplicate Emails](#) - delete keeping the lowest id, self-join
- [184. Department Highest Salary](#) - per-group max with a correlated subquery

**Where these come from:** [GFG DBMS interview questions](#) and [Set 2 \(SQL queries\)](#), cross-checked against recent off-campus experiences.

## 26. Interview FAQ bank - 45 questions

Quick-recall once the model answers above are solid. One or two lines each; if you can answer all 45 fast, your DBMS fundamentals are ready.

| Question                              | Answer                                                                                                 |
|---------------------------------------|--------------------------------------------------------------------------------------------------------|
| What is a DBMS?                       | Software that stores, queries, and manages data with concurrency control, integrity, and recovery.     |
| DBMS vs RDBMS?                        | An RDBMS stores data as related tables with keys and supports SQL; a DBMS is the broader term.         |
| What is a schema?                     | The structure or blueprint of the database - tables, columns, types, and constraints.                  |
| Three-schema architecture?            | Physical (storage), logical (tables and links), view (per-user slice).                                 |
| What is data independence?            | Changing one schema level without changing the level above it.                                         |
| Super key vs candidate key?           | A candidate key is a minimal super key - no attribute can be dropped.                                  |
| Primary key vs unique key?            | Primary key is not null and one per table; unique key allows a null and many per table.                |
| What is a foreign key?                | A column referencing another table's candidate (usually primary) key, enforcing referential integrity. |
| What is a functional dependency?      | $X \rightarrow Y$ means knowing X fixes the value of Y.                                                |
| Armstrong's axioms?                   | Reflexivity, augmentation, transitivity - sound and complete rules to derive all FDs.                  |
| What is attribute closure?            | The set of all attributes determinable from a given set; if it covers all, the set is a super key.     |
| Why normalize?                        | To remove redundancy and the insert, update, and delete anomalies.                                     |
| Define 1NF.                           | Every cell holds a single atomic value - no repeating groups.                                          |
| Define 2NF.                           | 1NF and no non-key column depends on part of a composite key (no partial dependency).                  |
| Define 3NF.                           | 2NF and no non-key column depends on another non-key column (no transitive dependency).                |
| Define BCNF.                          | For every non-trivial FD $X \rightarrow Y$ , X is a super key; stricter than 3NF.                      |
| Define 4NF.                           | BCNF and no non-trivial multi-valued dependency.                                                       |
| Partial vs transitive dependency?     | Partial = depends on part of a composite key; transitive = depends on a non-key column.                |
| What is denormalization?              | Deliberately adding redundancy to cut joins and speed reads, accepting extra update cost.              |
| What is a transaction?                | A group of operations treated as one all-or-nothing unit.                                              |
| Expand ACID.                          | Atomicity, Consistency, Isolation, Durability.                                                         |
| What does Atomicity guarantee?        | All operations complete or none do; failure triggers rollback.                                         |
| What does Durability guarantee?       | Committed changes survive crashes, via write-ahead logging.                                            |
| Transaction states?                   | Active, partially committed, committed, failed, aborted.                                               |
| What is a schedule?                   | An ordering of the interleaved operations of concurrent transactions.                                  |
| Serial vs concurrent schedule?        | Serial runs transactions one after another; concurrent interleaves them.                               |
| What is serializability?              | A concurrent schedule whose result equals some serial order.                                           |
| Conflict vs view serializability?     | Conflict is testable via the precedence graph; view is a looser superset and NP-hard to test.          |
| How to test conflict serializability? | Build the precedence graph; acyclic means conflict-serializable.                                       |
| What is a recoverable schedule?       | No transaction commits before the one whose data it read commits.                                      |

|                                            |                                                                                                         |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>What is a cascading abort?</b>          | One transaction's abort forces others that read its data to abort too.                                  |
| <b>What is 2PL?</b>                        | Two-phase locking: a growing phase that only acquires locks, then a shrinking phase that only releases. |
| <b>Does 2PL prevent deadlock?</b>          | No - it guarantees serializability, not deadlock-freedom.                                               |
| <b>Strict vs rigorous 2PL?</b>             | Strict holds all exclusive locks till commit; rigorous holds all locks till commit.                     |
| <b>Shared vs exclusive lock?</b>           | Shared is for reading (many allowed); exclusive is for writing (one only).                              |
| <b>What is timestamp ordering?</b>         | Each transaction gets a timestamp; operations are forced into timestamp order, late ones restart.       |
| <b>Wait-Die vs Wound-Wait?</b>             | Both use timestamps; Wait-Die makes the younger die, Wound-Wait makes the older preempt.                |
| <b>What is a dirty read?</b>               | Reading another transaction's uncommitted change that may be rolled back.                               |
| <b>Non-repeatable vs phantom read?</b>     | Non-repeatable = a row's value changes on re-read; phantom = the set of matching rows changes.          |
| <b>Default isolation level?</b>            | Most databases use Read Committed by default.                                                           |
| <b>Which level prevents all anomalies?</b> | Serializable.                                                                                           |
| <b>What is an index?</b>                   | A side structure that locates rows without a full scan; speeds reads, slows writes.                     |
| <b>B-tree vs B+ tree?</b>                  | B+ tree keeps data only in linked leaves (fast range scans); B-tree keeps data in internal nodes too.   |
| <b>Clustered vs non-clustered index?</b>   | Clustered sorts the table physically (one per table); non-clustered is a separate pointer list (many).  |
| <b>Why not index every column?</b>         | Each index slows every write and uses space; index only what you query on.                              |

## 27. The 30-minute revision card

**Last 30 minutes before the round. Read only this.**

### ***Normal forms, one line each:***

- 1NF - atomic values, no repeating groups.
- 2NF - 1NF and no partial dependency (only matters with a composite key).
- 3NF - 2NF and no transitive dependency.
- BCNF - every determinant is a super key.
- 4NF - BCNF and no multi-valued dependency.

### ***ACID, one line each:***

- Atomicity - all or nothing.
- Consistency - valid state to valid state.
- Isolation - concurrent equals some serial order.
- Durability - committed survives a crash.

### ***The isolation grid (what each level still ALLOWS):***

- Read Uncommitted - dirty, non-repeatable, phantom.

- Read Committed - non-repeatable, phantom.
- Repeatable Read - phantom only.
- Serializable - none.

***One-liners that score:***

- 2PL = grow (acquire) then shrink (release); guarantees serializability, NOT deadlock-freedom.
- Conflict-serializable = precedence graph has no cycle.
- B+ tree = data in linked leaves, fast range scans, database default.
- Clustered index = table sorted physically, one per table; non-clustered = separate pointer list, many per table.
- Primary key = not null + unique + one per table.

***Two traps that cost you easy points:***

- Saying 2PL prevents deadlock - it does not.
- Saying a table can have many clustered indexes - only one.

## 28. Final word

---

**You have normalization worked step by step, ACID with examples, the isolation grid memorised, and the indexing trade-offs. A DBMS round often includes one normalization question, one transaction or isolation question, and one indexing question. You have already practised all three. Answer calmly and you will have cleared the DBMS portion of the round.**